# Gli Array Monodimensionali in C

Dichiarazione, accesso, operazioni, funzioni e array a tappo

LINGUAGGIO C | STRUTTURE DATI | PROGRAMMAZIONE



# Cos'è un Array Monodimensionale?

## Definizione

Un array monodimensionale è una **sequenza di variabili dello stesso tipo** memorizzate in posizioni di memoria contigue. Si dichiara con:

```
int voti[5];
```

Gli indici validi vanno da **0** a **dimensione−1**. Ogni cella occupa `sizeof(tipo)` byte e le celle sono fisicamente adiacenti in memoria.

## Schema visivo a celle

| Indice | [0] | [1] | [2] | [3] | [4] |
|---|---|---|---|---|---|
| Valore | 7 | 8 | 6 | 9 | 7 |

Accedere a `voti[2]` restituisce **6**. Tentare di accedere a `voti[5]` provoca un comportamento indefinito: il linguaggio C non esegue nessun controllo automatico sui limiti.

# Dichiarazione e Inizializzazione

Esistono **cinque modi** principali per dichiarare e inizializzare un array in C:

### 1 – Lista completa

```
int a[3] = {10, 20, 30};
```

Tutti i valori specificati esplicitamente.

### 2 – Dimensione dedotta

```
int b[] = {10, 20, 30};
```

Il compilatore conta gli elementi.

### 3 – Lista parziale

```
int c[10] = {1, 2, 3};
```

I rimanenti 7 elementi vengono azzerati automaticamente a 0.

### 4 – Tutto a zero

```
int d[8] = {0};
```

Modo rapido per azzerare l'intero array.

### 5 – Costante simbolica

```
#define N 5
int e[N] = {...};
```

Rende la dimensione facile da modificare ovunque.

Schema per `int c[10] = {1, 2, 3}`: le prime 3 celle ricevono i valori espliciti, le restanti 7 vengono azzerate automaticamente dal compilatore.

| Indice | [0] | [1] | [2] | [3] | [4] | [5] | [6] | [7] | [8] | [9] |
|---|---|---|---|---|---|---|---|---|---|---|
| Valore | 1 | 2 | 3 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

# Dichiarazione Senza Inizializzazione

## Cosa succede in memoria?

Quando si dichiara un array specificando solo la dimensione, come nell'esempio:

```
int v[5];
```

il compilatore alloca lo spazio necessario in memoria (ad esempio, **5 × sizeof(int) = 20 byte contigui**). Tuttavia, NON inizializza i valori di queste celle. Questo significa che esse conterranno qualsiasi dato si trovasse precedentemente in quelle locazioni di memoria.

## Contenuto delle celle

| Indice | [0] | [1] | [2] | [3] | [4] |
|---|---|---|---|---|---|
| Valore | ??? | ??? | ??? | ??? | ??? |

Questi "valori spazzatura" (*garbage values*) sono imprevedibili e dipendono dal contenuto precedente della memoria. Il linguaggio C non effettua alcuna inizializzazione automatica per le variabili locali dichiarate in questo modo.

## Attenzione: Comportamento Indefinito

Leggere il contenuto di un array non inizializzato porta a un **comportamento indefinito** (undefined behavior), che può causare errori imprevedibili o crash del programma.

## Soluzioni Consigliate

È fondamentale inizializzare sempre gli array prima del loro utilizzo. Puoi farlo con un ciclo `for` o usando la sintassi `= {0}` al momento della dichiarazione per azzerare tutte le celle.

## Unica Eccezione

Le variabili globali e quelle dichiarate `static` sono un'eccezione: vengono automaticamente inizializzate a zero dal compilatore.

# Struttura in Memoria

### Layout per `int v[4]`

| Indice | [0] | [1] | [2] | [3] |
|---|---|---|---|---|
| Valore | 10 | 20 | 30 | 40 |
| Offset | +0 | +4 | +8 | +12 |

`sizeof(int) = 4 byte` — ogni elemento occupa 4 byte e le celle si trovano in indirizzi consecutivi.

### Calcoli utili

**Numero di elementi:**

```
sizeof(arr) / sizeof(arr[0])
```

Questo calcolo funziona **solo nel blocco dove l'array è dichiarato**.

**Attenzione nelle funzioni:** passando un array a una funzione, `sizeof(arr)` restituisce la dimensione del **puntatore** (tipicamente 8 byte), non dell'intero array. Bisogna quindi passare la dimensione come parametro separato.

# Accesso e Scorrimento

Schema di accesso per indice su `int v[5] = {3, 7, 1, 9, 4}`:

| Indice | [0] | [1] | [2] | [3] | [4] |
|---|---|---|---|---|---|
| Valore | 3 | 7 | 1 | 9 | 4 |

## Operazioni di lettura e scrittura

- **Lettura:** `x = v[2];` → x vale 1
- **Scrittura:** `v[2] = 99;` → la cella [2] diventa 99
- **Indice espressione:** `y = v[i + 1];`

## Ciclo for standard

```
for (i = 0; i < N; i++) { ... }
```

Varianti: ordine inverso (`i = N-1; i >= 0; i--`), passo 2 (`i += 2`), scorrimento parziale.

## ⚠️ Errore tipico

L'accesso **fuori range** — ad esempio `v[5]` su un array di 5 elementi — provoca **comportamento indefinito**. Il linguaggio C non esegue nessun controllo automatico a runtime: il programma può crashare, restituire valori casuali o sembrare funzionare correttamente.

# Lettura da Tastiera e Validazione

## Lettura base con `scanf`

```
for (i = 0; i < N; i++) {
    scanf("%d", &arr[i]);
}
```

L'operatore `&` è obbligatorio: passa l'indirizzo della cella all'array dove `scanf` scriverà il valore.

## Validazione con flag

```
while (!valido) {
    scanf("%d", &arr[i]);
    if (arr[i] >= 1 && arr[i] <= 10)
        valido = 1;
    else
        printf("Valore non valido.\n");
}
```

### Regola del flag

La condizione di uscita dal ciclo `while` si gestisce tramite una **variabile flag** (es. `int valido = 0;`). Non usare `break` per uscire dal ciclo: il flag rende il codice più leggibile, facilita il debug e segue le buone pratiche didattiche della programmazione strutturata.

# Operazioni Fondamentali: Somma e Media

## Pattern accumulatore

```
somma = 0;
for (i = 0; i < N; i++)
    somma += arr[i];
```

Inizializzare sempre a 0 prima del ciclo.

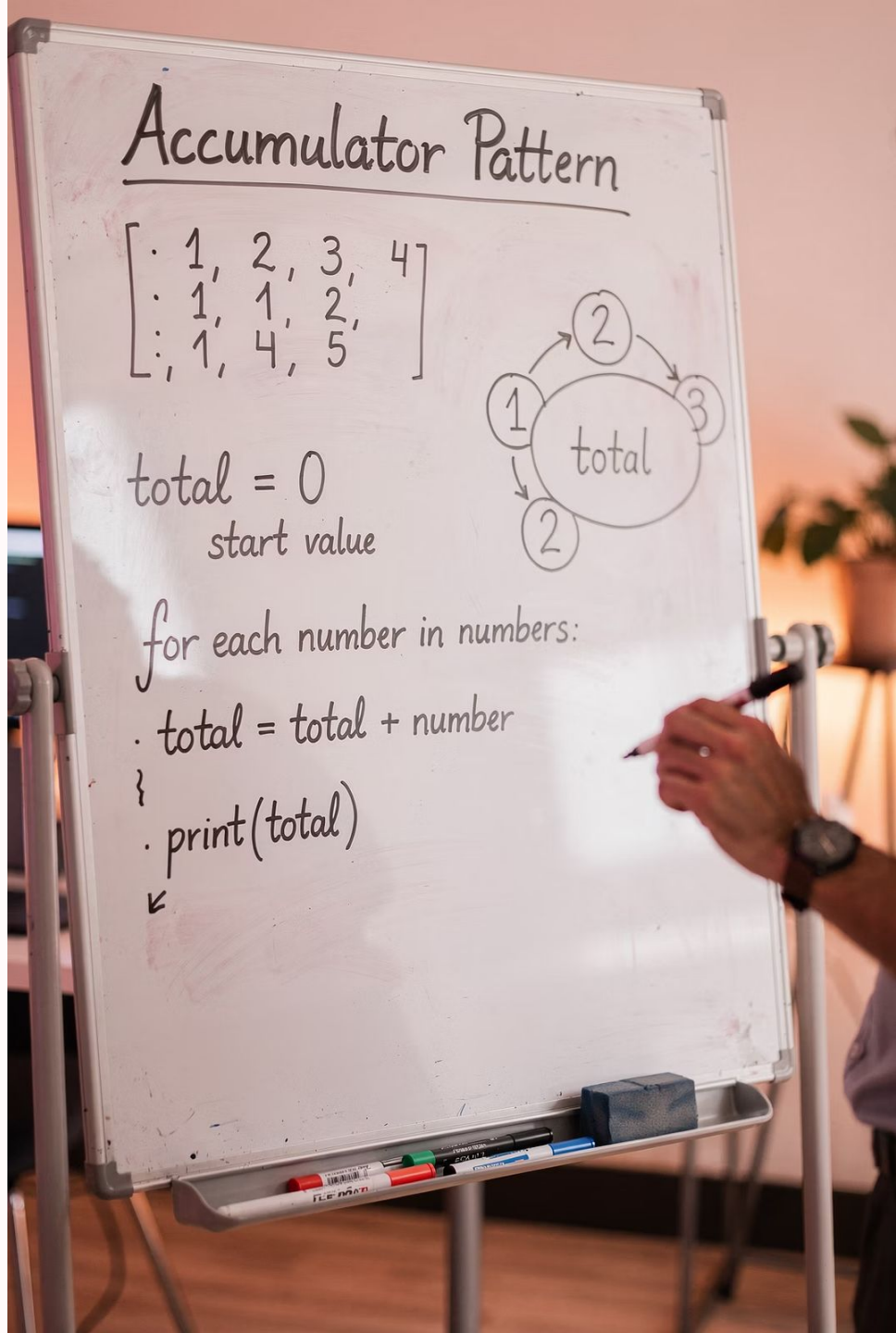
## Calcolo della media

```
media = (double)somma / N;
```

Il cast `(double)` è indispensabile: senza di esso la divisione sarebbe intera e si perderebbe la parte decimale.

## Varianti utili

- Somma solo dei positivi: `if (arr[i] > 0)`
- Somma per posizioni pari: `if (i % 2 == 0)`
- Somma cumulativa: stampa l'accumulatore a ogni passo



# Ricerca di Minimo, Massimo e Valore

Schema "ipotesi + smentita" su `int v[6] = {14, 7, 23, 5, 18, 9}`:

| Indice | [0] | [1] | [2] | [3] | [4] | [5] |
|---|---|---|---|---|---|---|
| Valore | 14 | 7 | 23 | 5 | 18 | 9 |

## Pattern "ipotesi + smentita"

- Inizializzare con il **primo elemento** (indice 0)
- Scorrere dal secondo elemento (indice 1)
- Se si trova un valore migliore, aggiornare
- Salvare l'**indice** (non solo il valore): permette di risalire alla posizione

## Ricerca con flag

```
while (arr[i] != cercato
       && !trovato) {
    i++;
}
```

Nessun `break`: usare il flag `trovato`. Per il **conteggio delle occorrenze** serve invece uno scorrimento completo senza uscita anticipata.

# Trasformazioni In-Place

Schema inversione con swap su `int v[6] = {A, B, C, D, E, F}`:

| | [0] | [1] | [2] | [3] | [4] | [5] |
|---|---|---|---|---|---|---|
| Prima | A | B | C | D | E | F |
| Dopo | F | E | D | C | B | A |

## Regole dello swap

- Il ciclo arriva solo a `N/2`: ogni coppia viene scambiata una volta sola
- Lo swap richiede una **variabile d'appoggio** `temp` per non perdere un valore

```
temp = v[i];
v[i] = v[N-1-i];
v[N-1-i] = temp;
```

## ⚠️ Copia di array

In C è **vietato** scrivere `arr2 = arr1`. La copia deve avvenire elemento per elemento con un ciclo:

```
for (i = 0; i < N; i++)
    arr2[i] = arr1[i];
```

Altre trasformazioni in-place: valore assoluto, azzeramento selettivo dei negativi, applicazione di formula su ogni elemento.

# Array e Funzioni: Passaggio per Riferimento

| Scalare (per valore) | Array (per riferimento) |
| --- | --- |
| main: x = 5 | main: v = [3, 7, 1, 9] |
| f(x): x_locale = 5 (copia) | f(v): stessa memoria dell'originale |
| x_locale = 999 → x in main: ancora 5 | v[0] = 99 → v in main: [99, 7, 1, 9] |

### Array → per riferimento

La funzione modifica l'array **originale**. Le modifiche sono visibili nel chiamante.

### Dimensione separata

La dimensione **non** viene passata automaticamente: serve un parametro `n`.

### Uso di `const`

`const int arr[]` indica che la funzione non modificherà l'array.

# Funzioni che Restituiscono Valori Aggregati

Il pattern comune: ricevono array + `n`, scorrono, restituiscono un **singolo valore** con `return`.

```c
int somma(const int arr[], int n) {
    int tot = 0;
    for (int i = 0; i < n; i++) tot += arr[i];
    return tot;
}

double media(const int arr[], int n) {
    return somma(arr, n) / (double)n;
}

int indice_massimo(const int arr[], int n) {
    int idx = 0;
    for (int i = 1; i < n; i++)
        if (arr[i] > arr[idx]) idx = i;
    return idx;
}

int cerca(const int arr[], int n, int val) {
    for (int i = 0; i < n; i++)
        if (arr[i] == val) return i;
    return -1;  /* non trovato */
}
```

**Restituire l'indice, non il valore:** permette al chiamante di accedere sia alla posizione che al dato. Il valore `-1` come sentinella di "non trovato" è una convenzione consolidata in C.

# Array a Tappo: Il Concetto

## Array a dimensione fissa (n=5)

| Indice | [0] | [1] | [2] | [3] | [4] |
|---|---|---|---|---|---|
| Valore | 10 | 20 | 30 | 40 | 50 |

```
for (i = 0; i < 5; i++) { ... }
```

## Array a tappo (TAPPO = -1)

| Indice | [0] | [1] | [2] | [3] | [4] | [5] | ... |
|---|---|---|---|---|---|---|---|
| Valore | 10 | 20 | 30 | -1 | ?? | ?? | ?? |

```
while (arr[i] != -1) { ... }
```

## Cos'è il tappo?

Un **array a tappo** usa un valore speciale detto **sentinella** per segnare la fine dei dati utili. Il ciclo si ferma quando incontra il tappo: non serve passare la dimensione `n`.

## Scelta del tappo

- `-1` per interi non negativi
- `0` per codici sempre positivi
- `'\0'` per char → le **stringhe C**!

# Operazioni con Array a Tappo

## Inserimento in fondo (prima del tappo)

| | | | | | |
|---|---|---|---|---|---|
| Prima | 10 | 20 | 30 | -1 | ?? |
| Dopo | 10 | 20 | 30 | 40 | -1 |

Il tappo si sposta di una posizione in avanti.

## Rimozione per indice (shift)

| | | | | | |
|---|---|---|---|---|---|
| Prima | 10 | 20 | 30 | 40 | -1 |
| Dopo | 10 | 30 | 40 | -1 | ?? |

Rimuovo posizione 1 (valore 20): gli elementi successivi shiftano a sinistra, trascinando con sé il tappo.

### ⚠️ Controllo della capacità

Prima di ogni inserimento verificare:

```
if (pos + 1 < MAX) {
    arr[pos] = nuovo_valore;
    arr[pos+1] = TAPPO;
}
```

L'array deve avere spazio per i dati **più il tappo**.
Dimenticare questo controllo porta a scrivere fuori dai limiti dell'array.

# Collegamento con le Stringhe C

Una stringa in C è semplicemente un **array di** `char` **con tappo** `'\0'`:

| Indice | [0] | [1] | [2] | [3] | [4] |
|---|---|---|---|---|---|
| `char s[] = "Ciao"` | C | i | a | o | \0 |

### Array int a tappo

```
while (arr[i] != -1) {
    /* elabora arr[i] */
    i++;
}
```

Tappo: `-1`

### Stringa C

```
while (str[i] != '\0') {
    /* elabora str[i] */
    i++;
}
```

Tappo: `'\0'` (ASCII 0)

**Stesso pattern, tappo diverso.** Capire gli array a tappo su interi significa capire il meccanismo alla base di tutte le stringhe C e delle funzioni come `strlen`, `strcpy`, `strcmp`.

# Errori Comuni da Evitare

### 1 – Indice fuori range

`for (i=0; i<=N; i++)` → quando `i == N` l'accesso è fuori limite. Usare sempre `<`, non `<=`.

### 2 – Array non inizializzato

`int a[5]; printf("%d", a[0]);` → valore indeterminato (spazzatura in memoria).

### 3 – Ciclo infinito nel while

Dimenticare di aggiornare l'indice (`i++`) dentro il `while` → il ciclo non termina mai.

### 4 – Indice unsigned decrescente

Usare `unsigned int` in un ciclo decrescente: non può diventare negativo, il ciclo va all'infinito.

### 5 – Array a tappo: tappo mancante

Dimenticare di posizionare il tappo dopo un inserimento, oppure non verificare che ci sia spazio prima di inserire.

# Riepilogo Concetti

| Operazione | Tecnica | Note |
|---|---|---|
| Dichiarazione | `int arr[N] = {...}` | Dimensione fissa, indici da 0 a N-1 |
| Scorrimento | `for (i=0; i<N; i++)` | Variante: `while` per condizioni complesse |
| Somma / media | Accumulatore + divisione float | Cast con `(double)` |
| Min / max | Ipotesi + smentita con indice | Inizializzare dall'elemento 0 |
| Ricerca | Flag `trovato`, niente `break` | Restituisce indice o -1 |
| Inversione | Swap con `temp`, ciclo a N/2 | In-place |
| Funzione | Passaggio per riferimento | Aggiungere `const` se non si modifica |
| Array a tappo | `while (arr[i] != TAPPO)` | Tappo: -1, 0 o '\0' |

CAPITOLO 1

# Esempi Pratici: Dichiarazione e Inizializzazione

Concetti base — tre file di esempio che esplorano la struttura fondamentale degli array.

## 1

### 010-dichiarazione-base.c

**Problema:** come si dichiara un array, come si assegnano i valori e come si stampano.

**Tecnica:** assegnazione per indice; stampa diretta e con ciclo `for`; calcolo del numero di elementi con `sizeof(arr)/sizeof(arr[0])`.

## 2

### 020-inizializzazione.c

**Problema:** quali sono i modi per inizializzare un array alla dichiarazione.

**Tecnica:** lista completa, dimensione dedotta, lista parziale con azzeramento automatico, `{0}` per azzerare tutto, `#define` come dimensione.

## 3

### 030-array-in-memoria.c

**Problema:** come è organizzato un array nella memoria del computer.

**Tecnica:** stampa degli indirizzi con `&arr[i]` e `%p`; calcolo dell'offset `i * sizeof(arr[0])`; verifica che la distanza tra celle consecutive sia sempre `sizeof(int)` byte.

CAPITOLO 1

# Esempi Pratici: Problemi su Dichiarazione

### 040 – Lancio del dado

Contare quante volte esce ogni faccia in 30 lanci. Array `frequenze[6]` come tabella di conteggio; indice: `frequenze[lancio-1]++`.

### 050 – Distribuzione voti

Classificare 20 voti in 6 fasce. Mappatura voto→indice con `if/else if`; istogramma percentuale con asterischi.

### 060 – Registro voti

Gestire voti di 8 studenti con nome e statistiche. Array paralleli `nomi[][]` e `voti[]`; minimo e massimo con salvataggio dell'indice per risalire al nome.

### 070 – Inventario magazzino

Gestire 6 prodotti con codice, nome, quantità e prezzo. Quattro array paralleli; valore calcolato al volo senza array aggiuntivo.

### 080 – Temperature settimanali

Analizzare temperature max di 7 giorni. Array di `double`; variazione consecutiva `temperature[i] - temperature[i-1]`; ciclo da `i=1`.

CAPITOLO 2

# Esempi Pratici: Accesso e Scorrimento

Concetti base — tre file che approfondiscono le modalità di accesso e lettura.

## 1

### 010-accesso-per-indice.c

**Problema:** mostrare tutte le forme di accesso a un elemento di un array.

**Tecnica:** indice costante, variabile ed espressione; verifica esplicita dei limiti con `if (indice >= 0 && indice < DIMENSIONE)`; passaggio di porzione con `&arr[i]`.

## 2

### 020-scorrimento-con-cicli.c

**Problema:** confrontare le tre forme di ciclo per scorrere un array.

**Tecnica:** `for`, `while` (con rischio ciclo infinito), `do-while`; scorrimento inverso; passo 2 con `i += 2`.

## 3

### 030-lettura-da-tastiera.c

**Problema:** leggere valori da tastiera con validazione.

**Tecnica:** `scanf("%d", &arr[i]);` validazione con flag `valido` e ciclo `while` annidato; barra grafica ASCII proporzionale al valore.

CAPITOLO 2

# Esempi Pratici: Problemi su Accesso e Scorrimento

### 040 – Rilevazioni pluviometriche

Analisi precipitazioni mensili di un anno. Mese più piovoso/secco con indice; barra proporzionale con `(int)(pioggia[i] / 5.0)` asterischi.

### 050 – Cassa automatica

Resto con minor numero di monete (greedy). Tagli in centesimi interi; `monete[i] = resto / tagli[i]`; aggiornamento residuo.

### 060 – Rubrica telefonica

Ricerca per prefisso nome e filtro per tipo. Array paralleli; doppio ciclo con flag; filtro su `telefoni[i][0] == '3'`.

### 070 – Sensori industriali

Dashboard di 8 sensori di temperatura con allarmi. Due cicli separati; due condizioni di allarme distinte (`< SOGLIA_MIN` e `> SOGLIA_MAX`).

### 080 – Questionario

Correggere risposta multipla con chiave. Due array di `char` paralleli; conversione minuscolo→maiuscolo con `c - 32`; punteggio percentuale.

CAPITOLO 3

# Esempi Pratici: Operazioni su Array

Concetti base — tre file che implementano le operazioni fondamentali.

| 1 | 2 | 3 |
| --- | --- | --- |
| **010-somma-e-media.c**<br>**Problema:** illustrare il pattern accumulatore nelle sue varianti principali.<br>**Tecnica:** accumulatore inizializzato a 0; somma condizionale con `if` (solo positivi, solo negativi); passo 2 per posizioni pari/dispari; somma cumulativa per osservare il valore crescere passo per passo. | **020-minimo-massimo-e-ricerca.c**<br>**Problema:** trovare minimo, massimo, secondo massimo e cercare un valore.<br>**Tecnica:** inizializzazione a `indice_max = 0` (non al valore); secondo massimo con condizione `i != indice_max`; ricerca con flag e `&& !trovato`; conteggio occorrenze con ciclo completo. | **030-trasformazione-elementi.c**<br>**Problema:** modificare il contenuto di un array in diversi modi.<br>**Tecnica:** inversione con swap e `temp`; ciclo di inversione fino a `N/2`; funzione `copia()` elemento per elemento; scarto dalla media come parametro per evitare ricalcoli. |

CAPITOLO 3

# Esempi Pratici: Problemi su Operazioni

### 040 – Andamento borsa

Analisi giornaliera titolo azionario su 10 giorni. Variazioni calcolate al volo senza secondo array; guadagno/perdita massima dalla prima variazione.

### 050 – Contatore caratteri

Occorrenze di ogni lettera in un testo. Indice con `c - 'a'`; scorrimento con `while (testo[i] != '\0')`; stampa solo lettere presenti.

### 060 – Classifica gara

Classifica di 8 atleti con distacco dal vincitore. Minimo (tempo più basso); `%02d` per i secondi; confronto con cast `(double)`.

### 070 – Paghe dipendenti

Paghe lorde di 6 dipendenti vs media aziendale. Array calcolato `paghe[i] = ore[i] * tariffe[i];` tre rami di confronto con la media.

### 080 – Rilevamento traffico

Top 3 strade più trafficate tra 8 senza ordinamento. Array `selezionato[]` come flag; ricerca ripetuta del massimo escludendo già scelti.

CAPITOLO 4

# Esempi Pratici: Array e Funzioni

Concetti base — tre file che dimostrano il passaggio di array a funzioni.

## 1

### 010-passaggio-array-a-funzione.c

**Problema:** mostrare la differenza tra passaggio per valore (scalari) e per riferimento (array).

**Tecnica:** `raddoppia(int arr[], int n)` modifica l'originale; `sizeof(arr)` dentro la funzione = dimensione del puntatore (8 byte); passaggio di porzione con `&valori[3]`.

## 2

### 020-funzioni-su-array.c

**Problema:** organizzare le operazioni fondamentali in funzioni riusabili.

**Tecnica:** `media()` richiama `somma()` internamente; tutte le funzioni di sola lettura dichiarano `const int arr[]`; `cerca()` restituisce `-1` come sentinella.

## 3

### 030-funzioni-che-modificano-array.c

**Problema:** incapsulare le trasformazioni in-place in funzioni riusabili.

**Tecnica:** `copia(src, dst, n)` dimostra che le modifiche a `dst` non toccano `src`; il `main` usa `copia()` prima di ogni trasformazione per ripartire dall'originale.

CAPITOLO 4

# Esempi Pratici: Problemi su Array e Funzioni

### 040 – Normalizzazione misure

Normalizzare tensione in [0.0, 1.0] e calcolare scarti. `normalizza(src, dst, n)` costruisce array di output; guardia contro massimo = 0.

### 050 – Cifrario Cesare

Cifrare e decifrare spostando ogni lettera di N posizioni. `cifra()` e `decifra()` in-place; formula cifratura con `% 26`; `+26` per evitare indici negativi.

### 060 – Gestione magazzino

Ricerca, carico e scarico prodotti. `trova_prodotto()` restituisce `-1`; `aggiorna_quantita()` rifiuta operazioni che porterebbero a quantità negative.

### 070 – Analisi partita

Statistiche marcatori squadra di calcio. `media_gol()` richiama `totale_gol()`; `stampa_classifica()` con top-k configurabile.

### 080 – Registro presenze

Presenze mensili di 6 dipendenti vs soglia. `percentuale_presenza(giorni, totale)` riusata da più funzioni; indice come valore di ritorno.

CAPITOLO 5

# Esempi Pratici: Array a Tappo

Concetti base — tre file che introducono e approfondiscono la tecnica del tappo.

| 1 | 2 | 3 |
|---|---|---|
| **010-introduzione-al-tappo.c**<br>**Problema:** confrontare direttamente la tecnica a dimensione fissa con quella a tappo.<br>**Tecnica:** `somma_con_n(arr, n)` vs `somma_con_tappo(arr)`; aggiunta manuale con `dati[pos] = 25; dati[pos+1] = TAPPO`; collegamento con le stringhe: `while (parola[j] != '\0')`. | **020-operazioni-con-tappo.c**<br>**Problema:** implementare le operazioni fondamentali su un array a tappo.<br>**Tecnica:** `lunghezza()` restituisce l'indice del tappo; `inserisci_in_fondo()` verifica `pos + 1 < MAX`; `rimuovi()` con shift include il tappo nel ciclo; riempimento progressivo da array vuoto `{TAPPO}`. | **030-tappo-su-array-di-char.c**<br>**Problema:** reimplementare le funzioni standard delle stringhe.<br>**Tecnica:** `lunghezza_stringa()` = `strlen`; `copia_stringa()` copia anche `'\0'`; `confronta_stringhe()` restituisce `a[i] - b[i]`; `concatena()` trova prima il `'\0'` di dst. |

CAPITOLO 5

# Esempi Pratici: Problemi su Array a Tappo

### 040 – Lista della spesa

Gestire una lista di prezzi a tappo. Prezzi in centesimi per evitare errori floating point; tappo = `-1`; conversione con `/ 100` e `% 100` nella stampa.

### 050 – Log eventi

Analisi log con classificazione per codice. Tappo = `0`; classificazione con `codice / 100` (divisione intera); scorrimento al contrario per l'ultimo evento.

### 060 – Coda prenotazioni

Coda FIFO di prenotazioni mediche con due array paralleli. Tappi sincronizzati su entrambi; shift parallelo; conversione minuti→HH:MM con `/ 60` e `% 60`.

### 070 – Sensori attivi

Registro dinamico dei soli sensori attivi. Tre valori di ritorno (1=OK, 0=pieno, -1=duplicato); `rimuovi()` usa `cerca()` per trovare l'indice; guardia `n > 0`.

### 080 – Playlist musicale

Inserimento in posizione e rimozione per ID. Shift in avanti dal fondo per inserimento; range valido di pos: [0, len]; controllo `len + 1 >= MAX`.